EEET2601 Engineering Computing 1

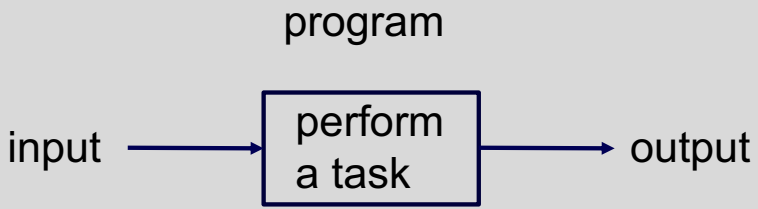
Basic Concepts

Quang Tran

RMIT UNIVERSITY

What is a program?

- A program is a **sequence of instructions** written for a computer to follow to perform a specific task
- A program is written in a programming language
- Types of instructions:
  1. Input
  2. Output
  3. Compute
  4. Control



C program structure

```
int main(void) {
    // Instructions for a computer to follow
    …
    return 0;
}
```

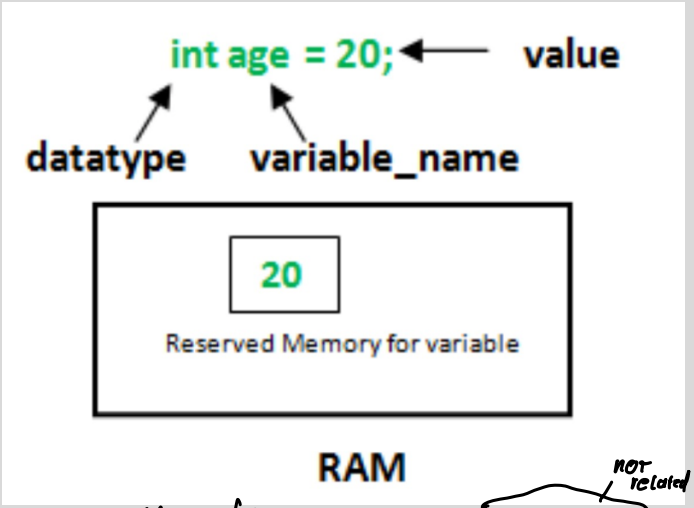
input of main FUNC

The `main` function returns an **exit status** of the program

- 0: exit success
- 1 (or other values): exit failure

Variable

- A computer program is to compute data, thus it must store data first
- A variable is a container to store a piece of data whose value can be changed during the execution of a program
- A variable has a name and stores a value of a data type



Variable

- Syntax:

```
data_type var_name;
data_type var1_name, var2_name;
```

- Examples:

```
int i;
int num1, num2;
```

Assignment statement

- An assignment statement is used to assign a value to a variable
- Syntax: `var_name = value;`
- Examples: initialize

```
int i;
i = 1;
i = (i + 4) * 2;
```

- A variable can be declared and assigned a value in one statement

```
int i = 1;
```

Constant

- A constant is a variable whose value cannot be changed
- Syntax:

```
const data_type CONST_NAME = value;
```

- Examples:

```
const float PI = 3.1416;
const int MAX_AGE = 130;
```

Comment

- Comments are notes in the program for human to read
- Comments are ignored by the computer
- Syntax:

```
// This is a one-line comment
/* This is
          a multi-line
   comment
*/
```

C naming convention

- Names are the heart of programming. Good names improve readability and reduce bugs.
- Names can be only made from **letters** (a-zA-Z), **digits** (0-9), and **underscore character** ('_')
- In this course, you must use **descriptive** yet **short** names that follow this C naming convention:

Type	Convention	Examples
Variable, function, file	Lowercase	num, getage, is_even, area
Constant	Uppercase	LIMIT, MAX_SCORE, EOF, AVG_SIZE

Data types

- A data type specifies the data storage format that a variable or constant can store.
- Fundamental data types: int, float, char
  - int: to store an integer. E.g. 5, 0, -10.
  - float: to store a floating-point number. E.g. 3.14, 62.0, -1.5.
  - char: to store an integer value in [0, 127] to represent a character in the ASCII Table. E.g. 63 for '?', 65 for 'A'.

Data types

- Other basic data types listed in the next slide are variants of the fundamental data types.
- There is no Boolean data type in C.
- 0 and 1 are used to represent FALSE and TRUE.
- The exact size of a data type depends on the computer architecture (64-bit, 32-bit, etc.)

Basic data types in a 64-bit architecture

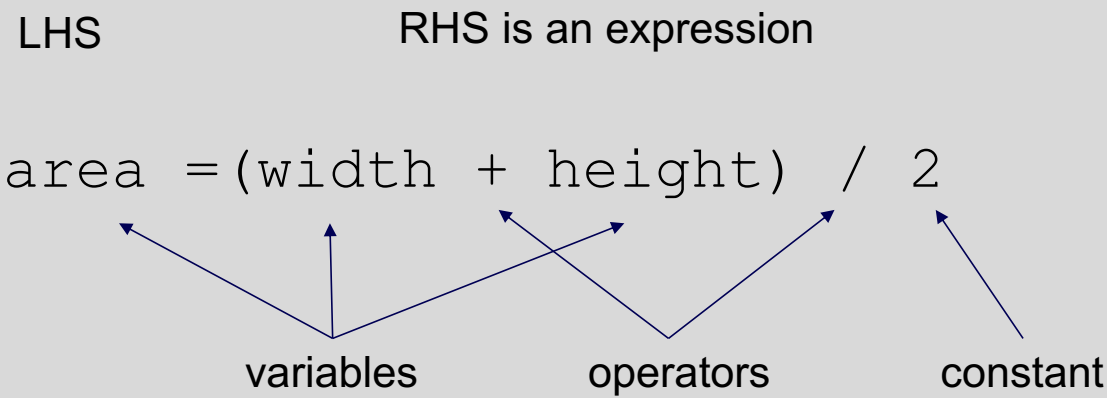
Data type	Size (bytes)	Range
char	1	-128 to 127
short	2	-32,768 to 32,767
int	4	-2,147,483,648 to 2,147,483,647
long	8	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned char	1	0 to 255
unsigned short	2	0 to 65,535
unsigned int	4	0 to 4,294,967,295
unsigned long	8	0 to 18,446,744,073,709,551,615
float	4	-3.4E38 to 3.4E38 (precision up to 6 decimal places)
double	8	-1.7E308 to 1.7E308 (precision up to 15 decimal places)

What is unsigned?

Demo

Write a program that calculates the area of a circle with a fixed radius of 10 cm

Operators and expressions



Operators and precedence

Level	Category	Operators	Associativity
1	Postfix	(), [], ->, ., ++, --	Left to right
2	Unary	+, -, !, ~, ++, --, (type), *, &, sizeof	Right to left
3	Multiplicative	*, /, %	Left to right
4	Additive	+, -	Left to right
5	Relational	<, <=, >, >=	Left to right
6	Equality	==, !=	Left to right
7	Assignment	=, +=, -=, *=, /=, %=	Right to left
8	Logical	&&, ||, !	Left to right

Basic input & output

- The standard library stdio.h provide some functions to input data into a program and output data from a program

Input	Output
int getchar()	int putchar(int char)
int scanf(const char *format, …)	int printf(const char *format, …)

- To use a function in stdio.h, you must include the library at the beginning of your program:

```
#include <stdio.h>
```

Examples

```
int ch = getchar();
putchar(ch);

int age;
printf("How old are you? ");
scanf("%d", &age);
printf("Your age is %d\n", age);

printf("Enter a character and an integer: ");
scanf("%c %d", &ch, &age);
```

address

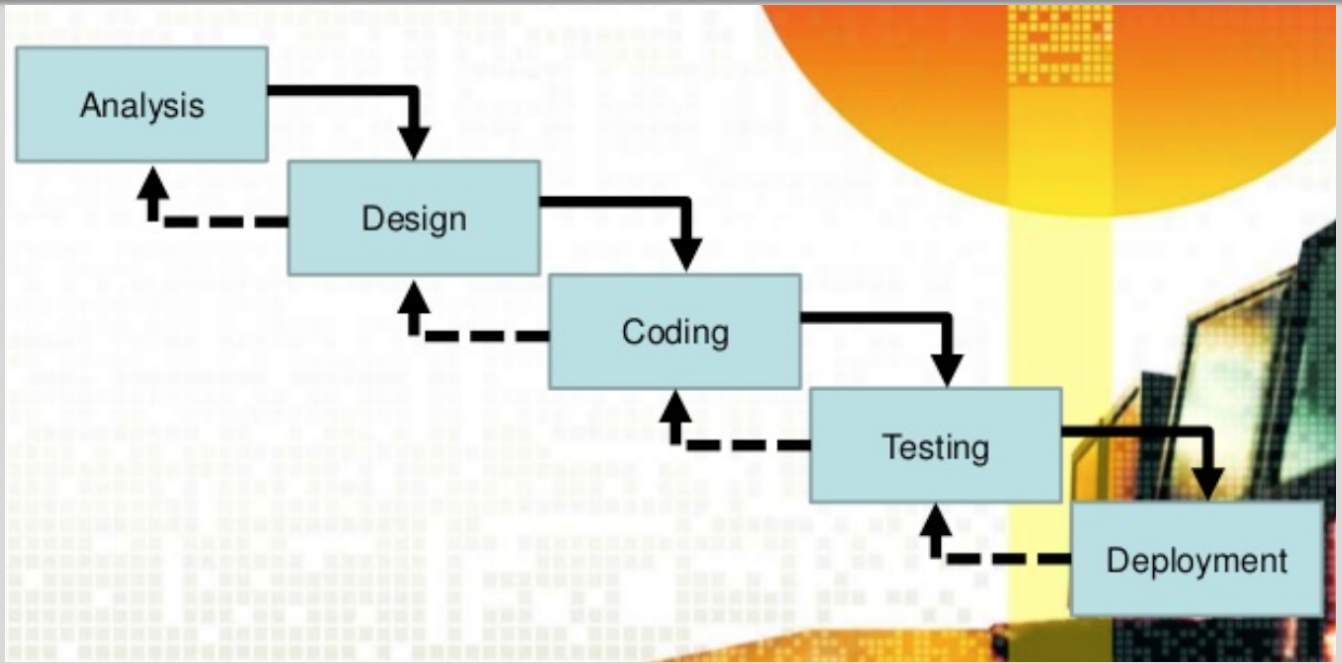
Format specifiers in printf() & scanf()

Format specifier	How the corresponding argument is written or read
%d (or %i)	Integer in decimal
%o	Integer in octal
%x	Integer in hexadecimal
%f	Floating-point (float)
%lf	Floating-point (double)
%e	Floating-point in exponential notation
%g	In the f-format or e-format, whichever is shorter
%c	Character
%s	String
%p	Pointer (memory address)

Demo

Write a program that gets a radius and calculates the area of the circle with that radius

Waterfall software development model



Waterfall software development model

1. **Analysis**: analyze a problem to figure out
   - *What* are the inputs?
   - *What* are the outputs?
   - *What* to compute?
   - *What* are the constraints on inputs, outputs, computation, etc.?
2. **Design**: come up with an algorithm to solve the problem
3. **Coding**: convert the algorithm to a program
4. **Testing**: check for errors
5. **Deployment**: make the program available for end-users

Algorithm

- An **algorithm** is a **sequence of steps** to perform a specific task
- Each step must be **simple**, **short** and **clear**
- If you have a complex step, decompose it to several simple ones
- Two ways to describe an algorithm:
  - ✓ Text
  - ✓ Flowchart

Algorithms in life

- A cooking recipe
- Google search
- Facebook newsfeed
- Google Maps direction
- Uber dynamic vehicle rerouting
- Bus arrival times
- Crime monitoring

Example: Make a Swiss breakfast

- Prep and cook time: 10 mins
- Ingredients:
  - 1 cups rolled oats
  - 1/4 tsp cinnamon
  - 2 TBS raisins
  - 2 TBS sliced almonds
  - 2 TBS dried apricots, chopped
  - 2 TBS sunflower seeds
  - 1 date, chopped
  - 1 TBS dried cranberries



Example: Make a Swiss breakfast

- Directions:
  1. Mix all ingredients together.
  2. Bring 2 cups water to a boil.
  3. Add mixed ingredients.
  4. Turn heat to low and cook uncovered, stirring occasionally until water is absorbed, about 7 minutes.
  5. Cover, and set for about 2 minutes before serving.
  6. Serve with low-fat milk or dairy-free milk.

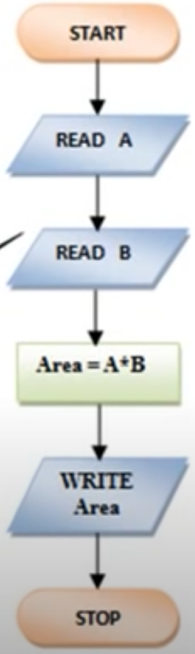
Example: Finding area of a rectangle

Finding Area of the rectangle

Algorithm

1. Start
2. Read Side length, A
3. Read side Length B
4. Area = A*B
5. Print or display Area
6. Stop

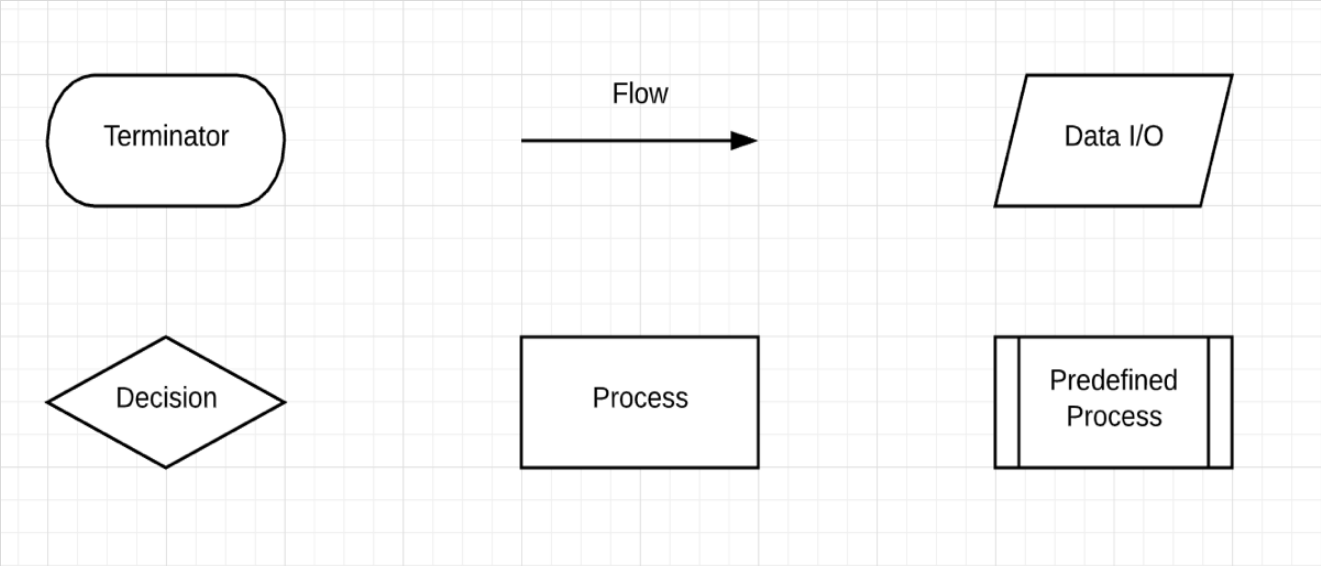
Flowchart



What makes a good algorithm?

- **Comprehensibility**: clear and precise so that it can be understood by other people (helps human efficiency)
- **Correctness**: perform the task without errors
- **Efficiency**: use as little time and as little space (memory) as necessary

Basic flowchart symbols



Basic flowchart symbols

- **Terminator** is an oval representing the start or end point of a flowchart
- **Arrows** show the direction of the chart's flow
- **Data I/O** is a parallelogram that indicates data input or output
- **Decision** is a diamond that indicates a condition to be tested
- **Process** is a rectangle that represents an action
- **Predefined Process** is a rectangle with an additional vertical line on the left and right end. It represents a **complex** action which should be then refined in another flowchart with more details

Exercise

Design an algorithm to get a temperature in Celsius and convert it to Fahrenheit

Algorithm:

1. Start
2. Get temperature in Celsius, celsius
3. fahrenheit = 32 + 1.8 * celsius
4. Print fahrenheit
5. Stop

References

1. Fresh2Refresh, C Programming Tutorial, 2019.
2. StudyTonight, C Programming Language Tutorial, 2019.
3. LucidChart, Flowchart Tutorial, 2019.
4. Jumail bin Taliba, Introduction to Flowcharting, 2019.

1.4.36

9:30 - 9.30

sas.rmit.edu.vn

learning lab

compile: gcc main.c -omain -lm